

予選 解説

2008年12月23日・情報オリンピック日本委員会

第8回日本情報オリンピック 予選

問題 1 タイムカード

配点 20点

時間制限 10 sec

メモリ制限 256 MiB

解説

この問題は、肩慣らしとJOI予選の出題形式に慣れてもらうことを意図して出題されている。

基本的には、退社時刻から出社時刻を引き算すれば良い。ただし、繰り下がりの処理を行わなければならない。秒、分、時という順に、引き算を行い、マイナスになれば繰り下がりをしてやればよい。

また、出社時刻と退社時刻をそれぞれ0時0分0秒からの経過秒数に直して引き算し、 在社時間を秒で計算して、それを時間・分・秒の形式に直すというような方法も考えられる。

原文：https://www2.ioi-jp.org/joi/2008/2009-yo-prob_and_sol/2009-yo-t1/review/2009-yo-t1-review.html

問題 2 コンテスト

配点 20点

時間制限 10 sec

メモリ制限 256 MiB

解説

この問題を解くには次のようにいくつかの方法が考えられるが、基本的に、ループなどのプログラムが書けるかどうかということを意図して出題されている。

考えられる方法の1つは、得点データを頭から見てゆき、そこまでの上位3人の得点を記憶しておく方法である。この方法の場合、新しいデータに対して、

- それが現在1位の得点より大きいなら、現在1,2位の得点がそれぞれ新たな2,3位になり、新しいデータが1位の得点になる。
- それが現在2位の得点より大きいなら、現在2位の得点が新たな3位になり、新しいデータが2位の得点になる。1位は変わらない。
- それが現在3位の得点より大きいなら、新しいデータが3位の得点になる。1,2位は変わらない。

という処理をすることになる。

得点データに現れる各得点が、その大学内で何位なのかを求める方法も考えられる。この場合、同じ大学内のすべての得点データのうち、注目したデータより大きい得点がいくつあるかを数えればよい。ただし、同点の場合にそれらを重複して数えたり数えもらしたりしないように、たとえば、「同点の場合は、データの中で先に現れる人の方を上位とする」などと決めておく必要がある。

また、ソート(データを大きい順や小さい順などに並べ替える処理)をするプログラムが書ける人は、10人分のデータをソートした後、上から3人の点数を足すという方法を使ってもよい。

原文：https://www2.ioi-jp.org/joi/2008/2009-yo-prob_and_sol/2009-yo-t2/review/2009-yo-t2-review.html

問題 3 連鎖

配点 20点

時間制限 10 sec

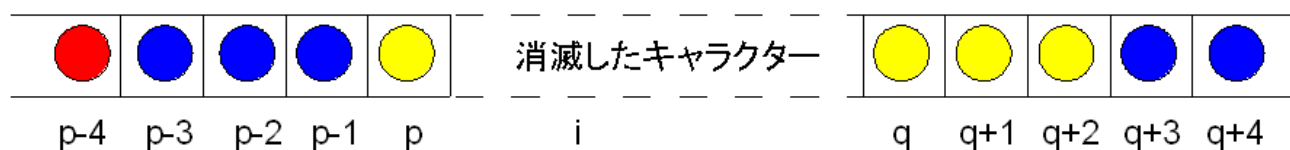
メモリ制限 256 MiB

解説

上から i 番目 ($1 \leq i \leq N$) のキャラクターを場所 i のキャラクターと呼ぶことにする。場所の i のキャラクターを単に「場所 i 」と呼ぶこともある。全ての場所 i と全ての色 c ($1 \leq c \leq 3$) に対して、初期状態において場所 i の色を c に変更した場合に消滅しないで残るキャラクターの個数を計算し、それらの最小値を求めれば良い。そのためには、場所 i の色を c に変更した場合に消滅するキャラクターの個数 $no(i, c)$ を求める関数を実装すればよい。

$no(i, c)$ を求めるには、まず、場所 i の色を c に変更することで、場所 i を含む連続する4つ以上のキャラクターが同じ色かどうかを確認する。同じ色が4つ以上連続する場合は、それらのキャラクターが消滅することにより新たに連続する4つ以上のキャラクターが同じ色になるのかどうかを確認する、ということ消滅するキャラクター数を数えながら繰り返せば良い。

キャラクターが消滅することにより新たに連続する4つ以上のキャラクターが同じ色になるのかどうかを確認する方法はいろいろ考えられるが、ここでは方針を1つ紹介しておこう。入力の2行目以降のキャラクターの色を配列に格納しているとする。下図は、場所 i の色を c に変更することにより、場所 $p+1$ から場所 $q-1$ までは消滅することが確認され、さらに場所 p および場所 q を含む4つ以上のキャラクターが同じ色になるかどうか確認しようとしている様子を表している。(場所 $p+1$ の色と場所 $q-1$ の場所は同じ色 c' で、場所 p の色は c' と異なるし、場所 q の色も c' とは異なる。)



消滅するキャラクター数を数える

場所 p と場所 q が異なる色ならこれ以上キャラクターは消滅しない。場所 p と場所 q が同じ色の場合、場所 p を含めて $p-1$ の方向に同じ色のキャラクターが連続している個数 s_1 と、場所 q を含めて $p+1$ の方向に同じ色のキャラクターが連続している個数 s_2 を求め、 $s_1 + s_1$ が4以上であればキャラクターの消滅するので、さらに消滅が続くか調べていけば良い。

最初に場所 i を色 c に変更する際と、両端の処理は特殊になるので、実装の際に注意すること。

本文は公式解説ページの表記を保持している。

原文：https://www2.ioi-jp.org/joi/2008/2009-yo-prob_and_sol/2009-yo-t3/review/2009-yo-t3-review.html

問題 4 薄氷渡り

配点 20点

時間制限 10 sec

メモリ制限 256 MiB

解説

ここでは、本問題を再帰を用いて解く方法を解説する。

まず最初に、薄氷が張られている広場をデータとして読み込まなければならない。これは、サイズが $(m+2) \times (n+2)$ となる 2 次元配列を用いると便利であろう。

たとえば、入力例 1 の場合だと、図1のようにデータを格納する。図1中、枠で囲まれた部分が配列に格納されているデータであり、また、黄色で示されている部分が入力ファイルから読み込んだデータである。周りを囲むように0が入力されているが、これはプログラム内で配列の端であるか否かを判断する手間を省くためである。

	0	1	2	3	4
0	0	0	0	0	0
1	0	1	1	0	0
2	0	1	0	1	0
3	0	1	1	0	0
4	0	0	0	0	0

図1：データを読み込んだ直後の配列

次に、再帰を用いて移動できる区画の最大値を求める方法の基本的なアイデアを説明する。たとえば、図1において(1,1)から薄氷を割り始めることを考えてみよう（図2の青い部分）。

	0	1	2	3	4
0	0	0	0	0	0
1	0	1	1	0	0
2	0	1	0	1	0
3	0	1	1	0	0
4	0	0	0	0	0

図2：(1,1)から割り始める場合

次に割る場所としては、(2,1)と(1,2)が考えられる。今回は、(1,2)を割ることにしよう。その結果、図3のようになる。

	0	1	2	3	4
0	0	0	0	0	0
1	0	0	1	0	0
2	0	1	0	1	0
3	0	1	1	0	0
4	0	0	0	0	0

図3：(1,1)を割り、(1,2)に移動した直後の様子

図3において、(1,1)の薄氷（赤い部分）は、割ってしまったので値が0となっている。また、今後、図3において(1,2)から薄氷を割り始める場合を考えれば良い。割れる枚数は、

すでに割った枚数（この例だと1枚） + 図3において(2,1)から薄氷を割り始めて移動可能な最大区画数

になる。

この説明の中で、図kにおいて(x,y)から薄氷を割り始める という表現が繰り返し出てきている。つまり、この部分は同一の処理を行っている。したがって、

入力：現在の薄氷のある区画を表す図k, 次に割り始める座標(x,y),すでに割った枚数depth

となる関数を作成し、関数内で繰り返し呼び出せばよい（再帰呼び出しをすればよい）。

東西南北に薄氷のある区画がなかったとき、再帰呼び出しは終了する。そのときの depth+1 が、そのルートをとった場合の区画数となる。グローバル変数に今までの最大値max_depthを保存しておき、max_depthよりdepth+1の方が大きければ、max_depthの値をdepth+1に更新する。

このような関数を作成し、すべての区画から始めることを考え、最後にmax_depthを出力すればよい。

次に、実際にプログラムを書くことを考えよう。入力が『現在の薄氷のある区画を表す図k, 次に割り始める座標(x,y),すでに割った枚数』である関数は、以下のような構造にすればよいだろう。ここで、グローバル変数max_depthには、今まで試してみた中で、一番多くの区画を通ったときの区画数を保存しておくことにする。

```

int check2(int table[N][M],int x, int y, int depth){

    table[x][y]=0; // 現在いる場所の薄氷を割る
    if(table[x-1][y]==1){ // 隣接する西側の区画に薄氷がある場合
        check2(table,x-1,y,depth+1); // そちらにすすんだ場合を考える
    }
    if(table[x+1][y]==1){ //隣接する東側の区画に薄氷がある場合
        check2(table,x+1,y,depth+1); // そちらにすすんだ場合を考える
    }
    if(table[x][y+1]==1){ // 隣接する南側の区画に薄氷がある場合
        check2(table,x,y+1,depth+1); // // そちらにすすんだ場合を考える
    }
    if(table[x][y-1]==1){ // 隣接する北側の区画に薄氷がある場合
        check2(table,x,y-1,depth+1); // // そちらにすすんだ場合を考える
    }
    table[x][y]=1; // 割った氷を元に戻しておく

    if((table[x][y+1]||table[x][y-1]||table[x+1][y]||table[x-1][y])==0){
        // 東西南北、どちらにも薄氷がない場合
        if(depth+1 > max_depth){
            // このパスを通して割った枚数が、
            // 今までたどった中で一番多くの区画を通っているならば、
            max_depth=depth+1; // 最長の区画数を更新する
        }
        return(0);
    }
}

```

最後に例として、この関数を、`check(table,1,1,0)` として呼び出した場合の推移を示す。

- `max_depth=0`とする。

`check(table,1,1,0)` が呼び出される

	0	1	2	3	4
0	0	0	0	0	0
1	0	1	1	0	0
2	0	1	0	1	0
3	0	1	1	0	0
4	0	0	0	0	0

- `table[1][1]=0` に

	0	1	2	3	4
0	0	0	0	0	0
1	0	0	1	0	0
2	0	1	0	1	0
3	0	1	1	0	0
4	0	0	0	0	0

- `check(table,2,1,1)` が呼び出される

	0	1	2	3	4
0	0	0	0	0	0
1	0	0	1	0	0
2	0	1	0	1	0
3	0	1	1	0	0
4	0	0	0	0	0

- 東西南北に薄氷のある区画がないので、再帰が終了する。`max_depth=2`に。

- `check(table,1,2,1)` が呼び出される

	0	1	2	3	4
0	0	0	0	0	0
1	0	0	1	0	0
2	0	1	0	1	0
3	0	1	1	0	0
4	0	0	0	0	0

- `table[1][2]=0` に

	0	1	2	3	4
0	0	0	0	0	0
1	0	0	1	0	0
2	0	0	0	1	0
3	0	1	1	0	0
4	0	0	0	0	0

- `check(table,1,3,2)` が呼び出される

	0	1	2	3	4
0	0	0	0	0	0
1	0	0	1	0	0
2	0	0	0	1	0
3	0	1	1	0	0
4	0	0	0	0	0

- `table[1][3]=0` に

	0	1	2	3	4
0	0	0	0	0	0
1	0	0	1	0	0
2	0	0	0	1	0
3	0	0	1	0	0
4	0	0	0	0	0

- `check(table,1,4,3)` が呼び出される

	0	1	2	3	4
0	0	0	0	0	0
1	0	0	1	0	0
2	0	0	0	1	0
3	0	0	1	0	0
4	0	0	0	0	0

- 東西南北に薄氷のある区画がないので、再帰が終了する．`max_depth=4`に。

- `table[1][3]=1` に

- `table[1][2]=1` に

- `table[1][1]=1` に

関数の終了

原文：https://www2.ioi-jp.org/joi/2008/2009-yo-prob_and_sol/2009-yo-t4/review/2009-yo-t4-review.html

問題 5 シャッフル

配点 20点

時間制限 10 sec

メモリ制限 256 MiB

解説

カードをシャッフルしていく様子をシミュレートする問題である。基本的な配列と繰り返しのプログラムが書ければ部分点を取ることは難しくない。しかし、満点を取るためには、アルゴリズムやデータ構造を工夫してプログラムを書く必要がある。

解法 1

配列を用いて n 枚のカードの状態を記憶する。例えば、長さ n の配列 $t[1], \dots, t[n]$ を用意して、一番上のカードに書かれた番号を $t[1]$ ，上から2枚目のカードに書かれた番号を $t[2]$ ，...，一番下のカードに書かれた番号を $t[n]$ に格納する。最初の山の状態では、 $t[1] = 1, t[2] = 2, \dots, t[n] = n$ である。

このようにしてカードの状態を記憶しておけば、配列を3段階に分けてコピーすることで、シャッフルを実現することができる。「シャッフル(x, y)」を行う場合は次のようにすればよい。作業用の配列 $s[1], \dots, s[n]$ を用意して、

$$\begin{aligned} s[1] &= t[y+1], \dots, s[n-y] = t[n] \\ s[n-y+1] &= t[x+1], \dots, s[n-x] = t[y] \\ s[n-x+1] &= t[1], \dots, s[n] = t[x] \end{aligned}$$

と代入する。これはループを3つ書くだけで実現できる。最後に配列 s の内容を配列 t にコピーすればよい。

この解法では、1回のシャッフルに $O(n)$ 時間かかるから、全体で $O(mn)$ 時間かかってしまう。この問題では n の値がとても大きいので、この解法では満点を取ることは難しい。

解法 2

この問題では、カードの枚数 n がシャッフルの回数 m に比べてとても大きいことに注目しよう。カードの山のうち、ほとんどの部分は連続して並んでいるはずである。したがって、カードの1枚1枚を別々のデータとして配列に記憶するのは無駄である。連続して並んでいるカードをまとめて、1つの「束」として処理することで、効率の良い解法が得られる。

例えば、次のようにすればよい。最初の山の状態では n 枚のカードが順番に並んでいるから、それを1つの「束」と考えて、 $[1, n]$ で表すことにする（ $[1, n]$ は『1 から n まで』という意味である）。同様に、カードの山の中に、番号 a から番号 b までの $b - a + 1$ 枚のカードが順番に並んでいたら、それは $[a, b]$ というカードの「束」としてまとめて処理する。

(次のようにイメージすれば分かりやすいかもしれない。 解法1では n 枚のカードを 1 枚ずつ別々に処理するので、膨大な時間がかかってしまう。 解法2では、連続したカードをクリップでまとめて処理する。もちろんシャッフルする際には、必要に応じてクリップをつけ直す手間がかかるが、カードの枚数が膨大な場合は、こうした方が効率が良い。)

この方法を問題文の入力例1に適用すると次のようになる。まず、最初の山の状態は、一つの「束」

[1,9]

で表される。「シャッフル(3,5)」を行うためには、まず、[1,9]という「束」を

[1,3] [4,5] [6,9]

という3つの「束」に分割する。そして、「束」を並び替えて、

[6,9] [4,5] [1,3]

とすればよい。後は、シャッフルを行う毎に、「束」を適切に分割して並び替えていけばよい。

シャッフルを行う毎に「束」の個数が増えていくから、1回のシャッフルは $O(m)$ 時間かかる。したがって、この解法では、全体で $O(m^2)$ 時間かかる。

入力データについて

参考までに、以下に、採点用の5つの入力データにおける n, m の値を挙げる。

1. $n = 20, m = 5$
2. $n = 500, m = 100$
3. $n = 500000000, m = 5000$
4. $n = 900000000, m = 5000$
5. $n = 1000000000, m = 5000$

解法1の場合、最初の2つのテストデータに対して正解を出力することができるが、 n の値が大きい残りの3つのデータに対しては、どんなに高速なPCを使ったとしても、現実的な時間内に正解を求めることはできないだろう。また、メモリが足りず、長さ n の配列を確保することができないかもしれない。

もちろん、解法2であれば、制限時間内に正解を出力することが可能である。

この問題のように、情報オリンピックでは、単純な解法で部分点を取ることは難しくないが、さらに高得点を得るためには問題を分析して、アルゴリズムやデータ構造を工夫する必要がある問題が多く出題される。ある程度プログラミングが上達してきたら、単に動けばいいや、という気持ちでプログラムを書くのではなく、『どこに無駄があるか。どうすれば効率の良いプログラムが書けるか』と考えながら書くとよい。そうすることで、プログラミングの面白さ・深さがますます実感できるようになるだろう。

原文：https://www2.ioi-jp.org/joi/2008/2009-yo-prob_and_sol/2009-yo-t5/review/2009-yo-t5-review.html

問題 6 ビンゴ

配点 20点

時間制限 10 sec

メモリ制限 256 MiB

解説

この問題はビンゴカード上の数字として表現されているが、実は n^2 個の数列 $\{a_1, a_2, \dots, a_{n \times n}\}$ として表現することができる。これ以降、この数列に関して考えることにする。

ここで数列にかけられている制限は M を超えない自然数であり、昇順の順列(小さいものから大きいものに順に並べられた数列)でかつ同じ数字は二度でないということと、その全ての合計が S であるということである。

解法 1

a_i の値と a_1 から a_i までの合計値との組み合わせからその種類数を引き出すことができる配列を持ち、それを元に a_{i+1} の値と a_1 から a_{i+1} までの合計値からその種類数を引き出すことができる配列を求める。これを i が 1 から $n \times n - 1$ まで繰り返し行う。ただし、この解法は一回の繰り返しの最大 $M \times M \times S$ の回数だけ計算する必要があるため、計算時間が長くなり途中までしか答えられないだろう。

解法 2

解法 2 では差分の数列 $b_i = a_i - a_{i-1}$ (ただし $b_1 = a_1$) なる数列を使う。数列の合計値が M を超えないことはすぐにわかるはずである。次にもう一つの条件、順列 a の合計値が S であることに注目する。 b_1 の値が 1 増えると合計値には $n \times n$ だけ影響が出るように、 b_i を 1 増やすと合計値が $n \times n - i + 1$ だけ増えることが少し考えるとわかるはずである。ここで b_1 から b_i までの合計値とその時点での a の合計値の組み合わせで解法 1 と同様に繰り返しを行う。一見このままでは解法 1 と同様の計算時間がかかるように思えるが、ある i での a の合計値の増え方は $n \times n - 1$ と一定であり、関数 $f_i(b \text{ の合計値}, a \text{ の合計値})$ を考えると

$$f_i(x, y) = f_i(x-1, y-(n \times n - i + 1)) + f_{i-1}(x-1, y-(n \times n - i + 1))$$

と書け、計算結果をそのまま利用できるので一回の繰り返しの最大 $M \times S$ の回数の計算ですみ、計算が素早く終わる。ただ、この方法は式を見てもすぐはわからないと思うので以下の表に a と b の関係を表で図示したので参考にしてほしい。

a_1	a_2	a_3	a_4	a_5	...	$a_{n \times n}$	合計値	使う回数
1	1	1	1	1	...	1	$n \times n$	b_1
0	1	1	1	1	...	1	$n \times n - 1$	b_2
0	0	1	1	1	...	1	$n \times n - 2$	b_3
0	0	0	1	1	...	1	$n \times n - 3$	b_4
0	0	0	0	1	...	1	$n \times n - 4$	b_5
:	:	:	:	:	:	:	:	:
0	0	0	0	0	...	1	1	$b_{n \times n - 1}$

上記より a_i が b_1 から b_i までの合計になるということがわかっていただけたでしょうか。この表を元に 1 から $n \times n$ までの数字をそれぞれ少なくとも 1 回ずつ使いかつ合計 M 回以下使いその合計値を S 以下にしたいと考えると、より実装も容易になるだろう。

原文：https://www2.ioi-jp.org/joi/2008/2009-yo-prob_and_sol/2009-yo-t6/review/2009-yo-t6-review.html

出典：情報オリンピック日本委員会「第8回日本情報オリンピック JOI 2008/2009 予選 問題・データ・解説」。本PDFは各解説ページを問題 1→2→3→4→5→6 の順に再構成したものです。

https://www2.ioi-jp.org/joi/2008/2009-yo-prob_and_sol/index.html

公式アーカイブでは、第5回以降の予選問題について CC BY-SA 4.0 の利用条件が案内されています。配点は原大会の成績集計（各問題20点）に合わせ、時間・メモリ制限は AtCoder の JOI 過去問表示を参照しました。公式ページの告知によると、問題3の入力データ No.3 と問題6の入力データ No.2 は問題文の記述を満たしていなかったため、採点対象から除外されました。